

Analisis de Patrones de Diseno y Arquetipos

Grupo Cordillera - Plataforma de Monitoreo Inteligente

1. Introduccion

Este documento describe los patrones de diseno y arquetipos seleccionados para la construccion de la Plataforma de Monitoreo Inteligente de Grupo Cordillera, justificando su uso en funcion de los problemas que resuelven.

2. Patrones de Diseno Implementados

2.1 Repository Pattern

Proposito: Abstraer la capa de persistencia de datos, proporcionando una interfaz uniforme para el acceso a datos.

Ubicacion: Todos los modulos (JPA Repository)

Implementacion:

- `JpaRepository<T, ID>` en cada entidad (Venta, Producto, Sucursal, Empleado, Departamento, Indicador, Usuario, Ticket)
- Custom repositories para consultas complejas (`VentaRepositoryCustom` + `VentaRepositoryImpl`)

Problema que resuelve: Desacopla la logica de negocio de los detalles de persistencia, permitiendo cambiar la implementacion de la base de datos sin afectar el codigo del servicio.

Beneficios:

- Codigo mas limpio y mantenible
- Facilidad para realizar pruebas unitarias (mocking del repositorio)
- Separacion de responsabilidades (Single Responsibility Principle)

2.2 Factory Method Pattern

Proposito: Definir una interfaz para crear objetos, pero permitir que las subclasses decidan que clase instanciar.

Ubicacion:

- `ms-datos-org/service/factory/EmpleadoFactory.java`
- `ms-indicadores/service/factory/CalculoIndicadorFactory.java`

Implementacion:

- `EmpleadoFactory` : Crea empleados con nombres, apellidos, cargos y departamentos aleatorios. Metodos `crearEmpleado()` y `crearEmpleadosMasivos()` .
- `CalculoIndicadorFactory` : Crea estrategias de calculo segun el tipo ("VENTAS", "INVENTARIO", "RENTABILIDAD") usando un switch expression.

Problema que resuelve: Centraliza la creacion de objetos complejos, evitando la duplicacion de logica de creacion y permitiendo agregar nuevos tipos sin modificar el codigo existente (Open/Closed Principle).

Beneficios:

- Reutilizacion de codigo
- Facil extension con nuevos tipos

- Configuración centralizada

2.3 Strategy Pattern

Propósito: Permitir que un algoritmo se seleccione en tiempo de ejecución.

Ubicación: `ms-indicadores/service/factory/`

Implementación:

- Interfaz `CalculoStrategy` con métodos `calcular()`, `getNombre()`, `getUnidad()`
- `CalculoVentasStrategy` : Calcula ticket promedio (total ventas / número transacciones)
- `CalculoInventarioStrategy` : Calcula rotación de inventario (productos vendidos / inventario promedio * 100)
- `CalculoRentabilidadStrategy` : Calcula margen de rentabilidad $((\text{ingresos} - \text{costos}) / \text{ingresos} * 100)$

Problema que resuelve: Permite cambiar el algoritmo de cálculo de KPIs sin modificar el código existente, siguiendo el principio Open/Closed.

Beneficios:

- Algoritmos intercambiables en tiempo de ejecución
- Cada estrategia es testeable de forma independiente
- Nuevos tipos de KPI se agregan creando nuevas implementaciones

2.4 Circuit Breaker Pattern

Propósito: Prevenir fallos en cascada cuando un servicio remoto no responde.

Ubicación: `api-gateway/` y `bff/` (Resilience4j)

Implementación:

- API Gateway: Circuit breakers configurados para rutas a microservicios con fallbacks
- BFF: `@CircuitBreaker` en clientes HTTP con métodos fallback que retornan datos vacíos

Problema que resuelve: Evita que el sistema completo falle cuando un microservicio está caído, proporcionando respuestas de respaldo (fallback) y permitiendo la recuperación gradual.

Beneficios:

- Tolerancia a fallos
- Degradación gradual del servicio
- Protección contra fallos en cascada

2.5 BFF (Backend For Frontend) Pattern

Propósito: Crear un backend específico para las necesidades del frontend.

Ubicación: Módulo `bff/`

Implementación: `DashboardService` agrega datos de 3 microservicios (ventas, indicadores, datos org) en un solo DTO (`DashboardDTO`).

Problema que resuelve: El frontend necesita datos consolidados de múltiples servicios; el BFF evita que el frontend tenga que hacer múltiples llamadas y expone exactamente los datos que necesita.

Beneficios:

- Reduccion de llamadas de red desde el frontend
- Datos optimizados para la interfaz de usuario
- Aislamiento de cambios en los microservicios internos

2.6 API Gateway Pattern

Proposito: Proporcionar un punto unico de entrada para todos los servicios backend.

Ubicacion: Modulo `api-gateway/`

Implementacion: Spring Cloud Gateway con enrutamiento basado en paths a los microservicios correspondientes, aplicando circuit breakers y time limiters.

Problema que resuelve: Centraliza el enrutamiento, la seguridad y las politicas de resiliencia, simplificando la configuracion del cliente.

Beneficios:

- Punto unico de entrada
- Enrutamiento centralizado
- Cross-cutting concerns (seguridad, logging, resiliencia)

2.7 Observer Pattern

Proposito: Definir una dependencia uno-a-muchos entre objetos para que cuando un objeto cambie su estado, todos sus dependientes sean notificados.

Ubicacion: `ms-ventas/event/`

Implementacion:

- `VentaRegistradaEvent` : Evento publicado cuando se registra una venta
- `VentaEventListener` : Escucha y registra en log los detalles de la venta
- `StockUpdateListener` : Verifica y registra el stock actualizado post-venta

Problema que resuelve: Permite que multiples componentes reaccionen a un evento (venta registrada) sin acoplar el emisor a los receptores.

Beneficios:

- Bajo acoplamiento entre componentes
- Extension facil (agregar nuevos listeners)
- Procesamiento asincrono de eventos

2.8 Builder Pattern

Proposito: Separar la construccion de un objeto complejo de su representacion.

Ubicacion: `ms-ventas/builder/VentaBuilder.java`

Implementacion: `VentaBuilder` permite construir objetos `Venta` paso a paso, configurando sucursal, usuario y detalles de producto con validacion en el metodo `build()`.

Problema que resuelve: Simplifica la creacion de objetos `Venta` que requieren multiples pasos de configuracion y validacion.

Beneficios:

- Codigo mas legible al construir objetos complejos

- Validación centralizada en el método build()
 - Inmutabilidad del objeto resultante
-

3. Arquetipos Maven

3.1 ms-service-archetype

Propósito: Arquetipo Maven para generar microservicios Spring Boot con JPA + PostgreSQL.

Estructura generada:

```
src/
├─ main/java/com/grupocordillera/{artifactId}/
│   ├── Application.java
│   ├── controller/SampleController.java
│   ├── entity/SampleEntity.java
│   ├── repository/SampleRepository.java
│   ├── service/SampleService.java
│   └─ exception/GlobalExceptionHandler.java
├─ main/resources/
│   ├── application.properties
│   └─ schema.sql
└─ test/java/.../ApplicationTests.java
```

Dependencias incluidas:

- Spring Boot Starter Web
- Spring Boot Starter Data JPA
- Spring Boot Starter Validation
- PostgreSQL Driver
- Lombok
- Spring Boot Starter Test

Uso:

```
mvn archetype:generate \
  -DarchetypeGroupId=com.grupocordillera \
  -DarchetypeArtifactId=ms-service-archetype \
  -DarchetypeVersion=1.0.0
```

3.2 bff-archetype

Propósito: Arquetipo Maven para generar módulos BFF con Spring Boot + Security + JWT + Resilience4j.

Estructura generada:

```
src/
├─ main/java/com/grupocordillera/{artifactId}/
│   ├── BffApplication.java
│   ├── config/DataInitializer.java
│   ├── config/RestTemplateConfig.java
│   └─ controller/SampleController.java
```

```

|   ├── dto/LoginRequest.java
|   ├── dto/LoginResponse.java
|   ├── entity/Usuario.java
|   ├── exception/GlobalExceptionHandler.java
|   ├── repository/UsuarioRepository.java
|   ├── security/JwtAuthFilter.java
|   ├── security/JwtUtil.java
|   ├── security/SecurityConfig.java
|   └── service/SampleService.java
└── main/resources/application.yml
└── test/java/.../BffApplicationTests.java

```

Dependencias incluidas:

- Spring Boot Starter Web
- Spring Boot Starter Security
- Spring Boot Starter Data JPA
- Resilience4j Circuit Breaker
- JWT (jjwt 0.12.6)
- H2 Database

4. Justificacion de la Seleccion

Criterios de Seleccion

Los patrones y arquetipos fueron seleccionados segun los siguientes criterios:

1. **Complejidad del dominio:** La plataforma requiere manejar ventas, empleados, indicadores y reportes, cada uno con logica especifica.
2. **Escalabilidad:** La arquitectura de microservicios permite escalar componentes de forma independiente.
3. **Mantenibilidad:** Los patrones de diseno facilitan la comprension y modificacion del codigo.
4. **Tolerancia a fallos:** El Circuit Breaker asegura que el sistema responda incluso cuando componentes fallan.
5. **Productividad del equipo:** Los arquetipos Maven estandarizan la creacion de nuevos componentes.

Matriz de Problemas vs Soluciones

Problema	Patron	Solucion
Acceso a datos acoplado	Repository	Abstraccion de persistencia
Creacion de objetos complejos	Factory Method	Creacion centralizada
Algoritmos variables	Strategy	Estrategias intercambiables
Fallos en cascada	Circuit Breaker	Degradacion gradual
Frontend necesita datos consolidados	BFF	Agregacion de datos
Multiples servicios, un punto de entrada	API Gateway	Enrutamiento centralizado
Reaccion a eventos del sistema	Observer	Notificacion desacoplada
Construccion de objetos complejos	Builder	Construccion paso a paso
Estandarizacion de proyectos	Maven Archetypes	Proyectos base reutilizables

5. Conclusion

La combinacion de 8 patrones de diseno y 2 arquetipos Maven proporciona una base solida para la Plataforma de Monitoreo Inteligente. La seleccion sigue principios SOLID y buenas practicas de arquitectura de microservicios, asegurando que el sistema sea escalable, mantenible y tolerante a fallos.